Software Engineering Lab

Lab 36

CORS & CSRF: Mengamankan Backend

Membedah Realitas CORS dan Pertahanan CSRF Multi-Layer di Sisi Server

Lukman / lukman-ss

Miskonsepsi Fatal: CORS Bukan Firewall

Banyak tim backend mengira CORS melindungi endpoint dari mutasi berbahaya

ASUMSI KELIRU

Menganggap CORS adalah tembok api backend penolak request asing
Meyakini penolakan header CORS otomatis menggagalkan serangan CSRF
Merasa aman hanya karena origin penyerang tidak ada di whitelist CORS

FAKTA LAPANGAN

Browser tetap mengeksekusi simple request POST langsung ke backend
Database memproses mutasi saldo sebelum browser mengevaluasi header respons
Penolakan CORS hanya memblokir pembacaan respons oleh JavaScript attacker

DAMPAK KERENTANAN

Transaksi finansial ilegal dieksekusi tanpa otorisasi sadar dari korban
Perubahan email dan password akun terjadi tanpa hambatan jaringan

Miskonsepsi bahwa CORS bertindak sebagai firewall backend masih sangat umum ditemukan. Padahal, CORS dirancang oleh browser murni untuk mengontrol izin pembacaan respons oleh JavaScript, bukan membatasi server dalam memproses request. Tanpa pertahanan CSRF di backend, penyerang dapat memicu mutasi state di database hanya melalui tag form HTML biasa.

CORS governs read permissions in JavaScript, not execution permissions on backend servers.

Mental Model: SOP vs CORS vs CSRF

Memisahkan pembatasan read, relaksasi browser, dan eksploitasi write

SAME-ORIGIN POLICY (SOP)

Membatasi JavaScript membaca data respons dari origin berbeda
Secara historis mengizinkan pengiriman request write (misal form submit)

CORS (RELAXATION MECHANISM)

Mekanisme negosiasi header HTTP untuk merelaksasi pembatasan SOP
Mengizinkan frontend membaca data respons dari API domain lain
Bukan mekanisme keamanan sisi server!

CSRF (WRITE EXPLOITATION)

Mengeksploitasi perilaku browser yang otomatis melampirkan cookie sesi
Penyerang memicu mutasi write tanpa perlu membaca data respons

Arsitektur keamanan web berdiri di atas tiga pilar ini. SOP melarang pembacaan respons lintas origin. CORS hadir sebagai mekanisme izin agar JavaScript dapat membaca data respons tersebut. Sementara itu, CSRF mengeksploitasi izin pengiriman data form yang secara historis diizinkan oleh SOP dengan memanfaatkan cookie sesi yang terlampir otomatis.

SOP restricts reading; CORS relaxes reading; CSRF exploits writing. Defend the write at the server.

Topologi Arsitektur Simulasi Bank

Membandingkan endpoint rentan vs endpoint terproteksi dalam ekosistem Go

ENDPOINT DISPATCH

GET /api/balance -> Cek saldo akun via cookie sesi
GET /api/csrf-token -> Endpoint penerbit token HMAC terikat sesi
POST /api/transfer/vulnerable -> Transfer rentan (hanya dibungkus CORS)
POST /api/transfer/protected -> Transfer aman (CORS + CSRF Token)
POST /api/transfer/fetch-metadata -> Transfer aman via Sec-Fetch-Site
POST /api/transfer/custom-header -> Transfer aman via X-Requested-With

ZERO DEPENDENCIES

Murni Go Standard Library (net/http, crypto/hmac, crypto/subtle, sync)
Membuktikan mekanisme otentikasi sesi dan penolakan payload secara presisi

KOMPONEN MODULAR

internal/cors: Evaluasi origin, header preflight OPTIONS, allow-credentials
internal/csrf: HMAC token generator, token validator, metadata middleware

Lab 36 mengimplementasikan bank service lengkap di internal/bank/app.go. Terdapat endpoint transfer yang sengaja dibiarkan rentan hanya dengan middleware CORS, serta varian endpoint terlindungi yang dibungkus oleh Anti-CSRF Token, Fetch Metadata, dan Custom Header middleware untuk membandingkan perilaku eksekusinya secara langsung.

Modular middleware isolates transport negotiation from state-changing business handlers.

Simple Request vs Preflight OPTIONS

Kapan browser meminta izin sebelum mengirim dan kapan langsung mengeksekusi

SIMPLE REQUEST (NO PREFLIGHT)

Method: GET, HEAD, atau POST
Content-Type: application/x-www-form-urlencoded, multipart, text/plain
Tanpa custom headers; browser LANGSUNG mengirim request ke backend
Target utama serangan formulir HTML CSRF klasik

PREFLIGHT REQUEST (OPTIONS)

Method: PUT, DELETE, PATCH atau Content-Type: application/json
Memakai custom header seperti X-CSRF-Token atau Authorization
Browser mengirim OPTIONS terlebih dahulu untuk meminta izin CORS
Jika preflight gagal, actual request TIDAK AKAN PERNAH dikirim

IMPLIKASI KEAMANAN

Penyerang CSRF memanfaatkan celah simple request agar server langsung mengeksekusi

Standar WHATWG Fetch membedakan request menjadi simple request dan non-simple request. Simple request menggunakan method standar dan encoding form tanpa custom header, sehingga dikirimkan langsung oleh browser tanpa konfirmasi awal. Sebaliknya, non-simple request memicu preflight OPTIONS. Penyerang CSRF memanfaatkan sifat simple request untuk mengeksekusi mutasi server seketika.

Simple requests fire immediately without asking; preflight requests ask permission first.

Anatomi Serangan Cross-Site Request Forgery

Bagaimana situs penyerang mengeksploitasi cookie korban yang tersimpan di browser

LANGKAH EKSPLOITASI

1. Korban login ke bank target dan memegang cookie sesi aktif di browser
2. Korban terpancing membuka tab situs penyerang (evil-attacker.com)
3. Halaman jahat menjalankan form submit tersembunyi via script JavaScript
4. Browser otomatis menyertakan cookie sesi korban ke request POST bank
5. Server backend memvalidasi cookie sesi dan mengeksekusi transfer dana

KENAPA CORS GAGAL MENCEGAH?

Server mengeksekusi mutasi data sebelum mengirimkan HTTP response
Browser memblokir pembacaan respons oleh penyerang karena origin mismatch
Penyerang tidak butuh membaca respons; saldo korban sudah terlanjur lenyap

KERUGIAN

Pemindahan aset finansial atau pembajakan email akun korban secara diam-diam

Serangan CSRF mengeksploitasi kepercayaan server terhadap browser korban. Ketika form submit otomatis dipicu dari situs penyerang, browser melampirkan cookie sesi milik korban. Server bank memproses transfer tersebut hingga tuntas. Walaupun browser kemudian menolak memberikan isi response kepada penyerang karena aturan CORS, mutasi saldo sudah selesai dicatat di database.

CSRF is blind execution: the attacker does not need to read the reply to steal the prize.

Bukti Empiris: CORS Tidak Mencegah Eksekusi

Test membuktikan saldo korban berkurang meskipun origin penyerang ditolak CORS

```
req := httptest.NewRequest(POST, "/api/transfer/vulnerable", body)
req.Header.Set("Origin", "https://evil-attacker.com")
req.AddCookie(&http.Cookie{Name: "session_id", Value: "session-victim-secret"})

app.ServeHTTP(rec, req)
// Response: 200 OK!
// ACAO Header: Kosong (ditolak untuk evil-attacker.com)
// Saldo Korban: Turun dari $1000 menjadi $700!
```

Pengujian TestIntegration_CORS_Does_Not_Prevent_CSRF_Execution membuktikan realitas ini secara definitif. Request POST dari origin evil-attacker.com berhasil memicu pemotongan saldo korban sebesar $300 dan menambah saldo penyerang. Meskipun middleware CORS tidak menyertakan header Access-Control-Allow-Origin, mutasi transaksi telah tuntas.

FAKTA TERVERIFIKASI

Status HTTP: 200 OK (handler bisnis transfer tetap tereksekusi)
Header Access-Control-Allow-Origin: Omitted (tidak diberikan ke attacker)
Saldo akun korban: Berkurang $300 ke rekening penyerang

Backend execution occurs before header evaluation. No CORS header does not mean no mutation.

labs/36-cors-and-csrf/tests/integration_test.go:54-87

Implementasi Middleware CORS Standar W3C

Menangani evaluasi origin, preflight OPTIONS, dan aturan kredensial

```
if !m.isOriginAllowed(origin) {
    if r.Method == http.MethodOptions {
        w.WriteHeader(http.StatusForbidden); return
    }
    next.ServeHTTP(w, r) // Non-preflight tetap diteruskan!
    return
}
if m.config.AllowCredentials {
    w.Header().Set("Access-Control-Allow-Origin", origin)
    w.Header().Set("Access-Control-Allow-Credentials", "true")
}
```

Di dalam internal/cors/middleware.go, kepatuhan terhadap spesifikasi W3C Fetch terlihat jelas. Ketika origin tidak terdaftar, request preflight OPTIONS langsung ditolak dengan HTTP 403. Namun, untuk request non-preflight (seperti simple POST), middleware harus meneruskan request ke handler berikutnya tanpa menyertakan header ACAO.

ATURAN SPESIFIKASI

Jika AllowCredentials: true, wildcard '*' dilarang keras oleh browser
Origin yang tidak ada di whitelist ditolak 403 HANYA pada preflight OPTIONS
Header Vary: Origin wajib disetel untuk mencegah cache poisoning di CDN

Standard CORS middleware allows unwhitelisted simple requests through to the next handler.

Aturan Kredensial & Wildcard Origin

Larangan spesifikasi: mengapa wildcard '*' ditolak saat membawa kredensial

LARANGAN SPESIFIKASI FETCH

Kombinasi ACAO: * dan ACAC: true dianggap ILEGAL oleh browser modern
Browser akan menolak memberikan respons ke JavaScript client
Mencegah kebocoran data terotentikasi secara tidak sengaja ke seluruh web

IMPLEMENTASI VALIDASI GO

TestCORS_Credentials_With_Wildcard_DisallowedInSpec memvalidasi aturan ini
Jika AllowCredentials aktif, server wajib merefleksikan origin spesifik pemanggil
Server tidak boleh memancarkan '*' jika request menyertakan cookie otentikasi

KESALAHAN UMUM

Merefleksikan Origin request secara membabi buta tanpa whitelist sama berbahayanya dengan wildcard

Spesifikasi web secara eksplisit melarang penggunaan Access-Control-Allow-Origin: * jika Access-Control-Allow-Credentials bernilai true. Jika server melanggar aturan ini, browser otomatis menolak respons. Backend wajib merefleksikan origin secara selektif hanya jika origin tersebut telah terverifikasi dalam whitelist aman.

Never pair wildcard origins with ambient credentials; explicit whitelisting is non-negotiable.

Arsitektur Pertahanan CSRF Multi-Layer

Menggabungkan 4 lapisan pertahanan untuk menjamin keamanan mutasi state

LAPISAN 1: SIGNED CSRF TOKEN

HMAC-SHA256 Token terikat Session ID, TTL, dan nonce (Double-Submit)

LAPISAN 2: FETCH METADATA

Evaluasi header Sec-Fetch-Site (tolak jika cross-site pada method mutasi)

LAPISAN 3: CUSTOM API HEADER

Wajibkan header kustom X-Requested-With (kebal dari submit form HTML)

LAPISAN 4: SAMESITE COOKIE

Konfigurasi cookie SameSite=Lax atau SameSite=Strict pada layer auth

Pertahanan CSRF yang tangguh tidak boleh bergantung pada satu mekanisme tunggal. Pendekatan defense-in-depth menggabungkan Signed Double-Submit Token berbasis HMAC, verifikasi Fetch Metadata dari browser, penegakan header custom untuk API AJAX, serta konfigurasi cookie SameSite yang ketat pada sisi klien.

Defense-in-depth: combine cryptographic tokens, browser metadata, and cookie boundaries.

Signed Double-Submit Token (HMAC-SHA256)

Membuat token stateless tanpa beban memori session store di server

STRUKTUR PAYLOAD TOKEN

> Payload: sessionID : timestamp : nonce
> Signature: HMAC_SHA256(secret_key, payload)
> Token Wire: base64(payload : base64(signature))

KEUNGGULAN POLA INI

Stateless: Server tidak perlu menyimpan token di Redis atau database
Cryptographically Bound: Terikat pada ID sesi pengguna aktif
Anti-Replay & Expiring: Dilengkapi batas waktu kedaluwarsa (TTL)

KEBAL SUBDOMAIN TAKEOVER

Naive Double-Submit cookie dapat disuntik attacker; HMAC signature menggagalkannya

Pola Signed Double-Submit Token menggabungkan efisiensi stateless dengan integritas kriptografis. Payload token berisi Session ID, timestamp pembuatan, dan random nonce 16-byte yang ditandatangani dengan HMAC-SHA256. Penyerang yang menguasai subdomain tidak dapat memalsukan token karena tidak memiliki secret key server.

HMAC-signed double-submit tokens provide stateless security immune to cookie injection.

Implementasi Generate & Validate Token

Validasi tanda tangan, Session ID, masa aktif, dan constant-time compare

```
parts := strings.Split(string(decoded), ":")
sessionID, tsStr, nonceStr, sigB64 := parts[0], parts[1], parts[2], parts[3]
if sessionID != expectedSessionID { return ErrInvalidToken }

ts, err := strconv.ParseInt(tsStr, 10, 64)
if err != nil || time.Since(time.Unix(ts, 0)) > tm.ttl {
    return ErrExpiredToken
}
expectedSig := hmac.New(sha256.New, tm.secret).Sum(payload)
if subtle.ConstantTimeCompare(actualSig, expectedSig) != 1 {
    return ErrInvalidToken
}
```

Method ValidateToken() di internal/csrf/token.go menjalankan validasi berlapis. Token diurai menjadi empat komponen. Validator memeriksa kecocokan Session ID, memastikan timestamp belum kedaluwarsa, dan menghitung ulang HMAC signature. subtle.ConstantTimeCompare digunakan untuk mencegah penyerang membongkar signature via timing attack.

ANTI-TIMING ATTACK

subtle.ConstantTimeCompare mengeliminasi kebocoran informasi via waktu eksekusi CPU

Constant-time verification prevents side-channel timing attacks on cryptographic signatures.

Middleware Proteksi CSRF di HTTP Handler

Mengabaikan method aman dan memvalidasi token dari header maupun form body

```
if r.Method == GET || r.Method == HEAD || r.Method == OPTIONS {
    next.ServeHTTP(w, r); return
}
cookie, err := r.Cookie(m.cookieName)
if err != nil || cookie.Value == "" {
    http.Error(w, "Unauthorized: missing cookie", 401); return
}
token := r.Header.Get("X-CSRF-Token")
if token == "" { token = r.PostFormValue("csrf_token") }

if err := m.tokenManager.ValidateToken(token, cookie.Value); err != nil {
    http.Error(w, "Forbidden: CSRF validation failed", 403); return
}
```

Middleware CSRF bertindak sebagai gerbang pengaman mutasi data. Request dengan method aman (GET, HEAD, OPTIONS) langsung dilewatkan. Untuk method mutasi (POST, PUT, DELETE), middleware mengekstrak cookie sesi dan mengambil token CSRF dari header atau form field. Jika token tidak cocok atau kedaluwarsa, server menolak dengan HTTP 403 Forbidden.

FLEXIBILITAS TRANSPORT

Mendukung AJAX SPA via header X-CSRF-Token
Mendukung form HTML tradisional via field csrf_token

Safeguard state mutation: exempt safe methods, extract token from header/form, verify against session.

Fetch Metadata: Memanfaatkan Sec-Fetch-Site

Header antipemalsuan browser untuk mengidentifikasi asal usul inisiasi request

NILAI SEC-FETCH-SITE

same-origin : Request diinisiasi dari origin yang sama persis (Aman)
same-site : Request diinisiasi dari subdomain dalam registable domain sama
cross-site : Request diinisiasi dari situs luar sepenuhnya (Waspada!)
none : Navigasi langsung oleh user (misal ketik URL di address bar)

```
fetchSite := r.Header.Get("Sec-Fetch-Site")
if fetchSite == "cross-site" {
    if r.Method != GET && r.Method != HEAD && r.Method != OPTIONS {
        http.Error(w, "Forbidden: Cross-site rejected", 403)
        return
    }
}
```

KEUNGGULAN UTAMA

Header disetel langsung oleh kernel browser; tidak dapat diubah oleh JavaScript frontend

Fetch Metadata merupakan standar W3C modern di mana browser melampirkan header Sec-Fetch-Site pada setiap request keluar. Header ini bernilai cross-site jika dipicu dari domain luar. Karena header ini dikelola langsung oleh mesin browser dan dilindungi dari manipulasi JavaScript, backend dapat menolak request mutasi cross-site tanpa perlu token sama sekali.

Browser-controlled metadata tells the unvarnished truth about request origin.

Custom Header Enforcement untuk API

Memanfaatkan keterbatasan form HTML standar untuk mematikan eksploitasi CSRF

MEKANIKA TEKNIS

Formulir HTML standar (
) TIDAK BISA menyetel custom HTTP headers
Hanya JavaScript terprogram (fetch / axios) yang dapat menambahkan custom header
Menambahkan custom header otomatis memaksa browser mengirim preflight OPTIONS

```
func RequireCustomHeaderMiddleware(headerKey, expectedValue string) {
    val := r.Header.Get(headerKey)
    if val == "" || !strings.EqualFold(val, expectedValue) {
        http.Error(w, "Forbidden: Missing custom header", 403); return
    }
}
```

BATASAN STRATEGI

Sangat efektif untuk REST API murni, namun tidak dapat dipakai pada aplikasi berbasis server-side rendered form

Salah satu pertahanan paling sederhana namun ampuh untuk API backend adalah mewajibkan header kustom seperti X-Requested-With: XMLHttpRequest. Karena tag form HTML standar di browser tidak memiliki kemampuan menambahkan custom header HTTP, seluruh upaya eksploitasi form CSRF klasik langsung gagal sebelum mencapai handler bisnis.

HTML forms cannot emit custom headers. Require a custom header, defeat form CSRF by definition.

Peran Atribut Cookie SameSite: Lax vs Strict

Membendung pengiriman kredensial otomatis pada batas navigasi browser

SAMESITE=STRICT

Cookie tidak pernah dikirim pada request lintas situs dalam kondisi apapun
Bahkan saat user mengklik link dari email atau search engine, cookie ditahan
Sangat aman, namun mengorbankan kenyamanan login navigasi eksternal

SAMESITE=LAX (DEFAULT BROWSER MODERN)

Cookie ditahan pada cross-site POST (memblokir serangan form CSRF)
Cookie tetap dikirim pada top-level GET navigation (misal klik link)
Wajib: Endpoint GET harus idempotent dan tidak boleh memutasi state!

SAMESITE=NONE

Cookie selalu dikirim lintas situs; wajib menyertakan flag Secure (HTTPS)

Atribut SameSite pada cookie menentukan kapan browser diizinkan melampirkan cookie sesi. SameSite=Lax yang menjadi default browser modern menahan cookie pada request POST lintas situs. Namun, karena cookie tetap dikirim pada navigasi top-level GET, sistem tidak boleh melakukan mutasi saldo atau perubahan data melalui method GET.

SameSite=Lax blocks cross-origin POST cookies; keep GET endpoints strictly read-only.

Eksekusi Demo: Bukti Terminal Langsung

Simulasi nyata serangan dan pertahanan pada cmd/demo/main.go

```
[1] Initial State: Victim: $1000 | Attacker: $50
[2] DEMO: Attack on Vulnerable Endpoint (Origin: https://evil.com)
    HTTP Response Code: 200 | CORS Header: ""
    [RESULT] Transfer EXECUTED! Victim: $600 | Attacker: $450
[3] DEMO: Attack on Protected Endpoint (No CSRF Token)
    HTTP Response Code: 403
    [RESULT] Transfer BLOCKED! Victim: $600 | Attacker: $450
[4] DEMO: Legitimate Flow with Valid CSRF Token
    Obtained Signed Token: c2Vzc2lvbi12aWN0aW0tc2Vjc...
    [RESULT] Transfer Succeeded! Victim: $500 | Attacker: $550
```

Program demo di cmd/demo/main.go memperlihatkan realitas serangan secara nyata. Pada endpoint rentan, serangan transfer $400 dari evil.com sukses dieksekusi menghasilkan saldo korban $600. Namun, ketika serangan diarahkan ke endpoint terlindungi tanpa token CSRF, server mengembalikan status 403 Forbidden dan saldo korban tetap aman.

Terminal output confirms: vulnerable endpoints fall to simple POST; protected endpoints hold firm.

Verifikasi Test Suite & Bebas Race Condition

13 test case diverifikasi 100% lulus dengan Go Race Detector aktif

```
PASS: TestBankTransfer_VulnerableEndpoint       (0.00s)
PASS: TestCORS_Preflight_Success                (0.00s)
PASS: TestCORS_Credentials_With_Wildcard        (0.00s)
PASS: TestTokenManager_GenerateAndValidate      (0.00s)
PASS: TestTokenManager_ExpiredToken             (0.02s)
PASS: TestIntegration_CORS_Does_Not_Prevent     (0.00s)
PASS: TestIntegration_CSRF_Token_Prevents       (0.00s)
PASS: TestIntegration_SecFetchSite_Protection   (0.00s)
PASS: TestIntegration_Concurrency_RaceCondition (0.00s)
PASS: go test -race ./... (Race-Clean Verified)
```

Seluruh 13 pengujian unit dan integrasi lulus dengan Go Race Detector aktif. Pengujian mencakup verifikasi penolakan origin CORS, validasi tanda tangan HMAC, penolakan token kedaluwarsa, blokade cross-site Sec-Fetch-Site, pencegahan token reuse lintas sesi, hingga stabilitas konkurensi di bawah beban paralel.

UJI KONKURENSI

20 worker simultan melakukan pembuatan dan validasi token tanpa tabrakan memori

Deterministic test assertions prove security invariants and concurrency safety.

Matriks Evaluasi: Mekanisme Pertahanan

Perbandingan komparatif 5 pendekatan keamanan web lintas origin

Mekanisme Keamanan	Target Proteksi	Lokasi Penegakan	Tangkal Form CSRF?	Ketergantungan Browser
CORS Middleware	Relaksasi Read	Browser & API	TIDAK	Penuh (Fetch Spec)
Signed CSRF Token	Mutasi Write	Backend Server	YA	Nol (Bekerja di semua)
Fetch Metadata	Mutasi Write	Gateway / App	YA	Butuh browser modern
Custom Header API	Mutasi Write	Gateway / App	YA	Nol (API Client)
Cookie SameSite	Ambient Cookie	Browser Engine	YA (Lax/Strict)	Butuh browser modern

Tabel komparasi memperjelas posisi masing-masing mekanisme. CORS sama sekali tidak menangkal form CSRF karena fokusnya adalah relaksasi pembacaan respons. Signed CSRF Token adalah satu-satunya mekanisme yang 100% independen dari peramban karena divalidasi langsung oleh logika kriptografis backend.

KESIMPULAN EVALUASI

Hanya Signed CSRF Token yang memberikan pertahanan 100% mandiri di sisi backend

Lapisan lain memperkuat pertahanan di layer browser dan network gateway

Only cryptographic tokens provide absolute backend independence; other layers reinforce the perimeter.

Ancaman Tersembunyi: XSS Melumpuhkan CSRF

Mengapa pertahanan CSRF terbaik sekalipun runtuh jika origin memiliki celah XSS

ALUR BYPASS VIA XSS

1. Penyerang menemukan celah Stored / Reflected XSS pada domain target
2. Script jahat penyerang dieksekusi di dalam context origin yang sah
3. Karena berada di origin yang sama, SOP mengizinkan script membaca DOM
4. Script melakukan fetch ke /api/csrf-token dan membaca token sah
5. Script mengeksekusi POST transfer lengkap dengan token dan cookie korban

IMPLIKASI KEAMANAN

XSS adalah kerentanan tingkat origin yang mengalahkan seluruh pertahanan CSRF
Pertahanan CSRF wajib didampingi Content Security Policy (CSP) dan sanitasi input

Penting untuk diingat bahwa seluruh pertahanan anti-CSRF (baik token, SameSite, maupun Fetch Metadata) runtuh jika aplikasi memiliki kerentanan Cross-Site Scripting (XSS). Jika penyerang dapat menyuntikkan script ke origin target, script tersebut memiliki hak istimewa SOP untuk membaca token CSRF secara sah dan mengeksekusi transaksi atas nama korban.

XSS invalidates all anti-CSRF defenses by executing within the trusted origin context.

Batasan Model & Implementasi Lab 36

Penyederhanaan pedagogis untuk menonjolkan prinsip mekanika protokol

BATAS 1: SECRET KEY IN-MEMORY

Kunci rahasia HMAC menggunakan byte slice statis dalam kode demo
Produksi wajib menggunakan AWS Secrets Manager, Vault, dan key rotation

BATAS 2: DELIMITER STRING PARSER

Token memisahkan field menggunakan karakter titik dua (':')
Jika sessionID sistem memuat karakter ':', parsing token akan terganggu
Produksi memerlukan format terstruktur seperti JSON, Protobuf, atau hex encoding

BATAS 3: LINGKUP BROWSER

SOP, CORS, dan CSRF murni berada pada lingkup runtime peramban web
HTTP client non-browser (cURL, script backend) tidak terikat aturan ini

Lab ini menyederhanakan beberapa aspek infrastruktur untuk menjaga fokus edukasi. Kunci rahasia HMAC di-hardcode dalam memori, dan token diurai menggunakan string delimiter sederhana. Dalam sistem produksi riil, kunci HMAC wajib dikelola melalui key management service dan token sebaiknya menggunakan format terstruktur seperti JSON Web Signature (JWS) atau Protobuf.

A lab isolates cryptographic mechanics; production demands secret management and structured payloads.

Checklist Audit Keamanan CORS & CSRF

9 langkah verifikasi wajib sebelum merilis backend ke lingkungan produksi

VERIFIKASI CORS & HTTP

[ ] Method GET, HEAD, OPTIONS dipastikan bebas dari mutasi data database
[ ] CORS whitelist hanya mencantumkan domain tepercaya (tanpa refleksi liar)
[ ] AllowCredentials: true TIDAK PERNAH dipadukan dengan origin '*'

VERIFIKASI ANTI-CSRF

[ ] Seluruh rute mutasi state (POST/PUT/DELETE) dilindungi Anti-CSRF Token
[ ] Token CSRF ditandatangani HMAC-SHA256, terikat Session ID, dan memiliki TTL
[ ] Validasi signature menggunakan subtle.ConstantTimeCompare
[ ] Cookie otentikasi dipasangi flag HttpOnly, Secure, dan SameSite=Lax
[ ] API gateway memverifikasi header Sec-Fetch-Site dan Custom Header
[ ] Aplikasi terlindungi dari celah XSS via Content Security Policy (CSP)

Gunakan checklist ini saat melakukan security audit pada sistem web Anda. Pastikan endpoint GET bebas dari efek samping mutasi data, jangan pernah menggabungkan wildcard CORS dengan kredensial, wajibkan validasi token HMAC yang terikat ke sesi pada setiap mutasi POST, dan perkuat perimeter dengan header Sec-Fetch-Site serta cookie SameSite.

Verify read-only GETs, strict origin whitelists, signed tokens, and CSP before shipping.

Tiga Prinsip Inti Keamanan Cross-Origin

Aksioma fundamental untuk melindungi backend dari eksploitasi browser

1. NEVER TRUST CLIENT BOUNDARIES FOR SERVER INTEGRITY

CORS adalah aturan kepatuhan browser, bukan pelindung integritas database
Validasi otorisasi dan mutasi data wajib ditegakkan sepenuhnya di sisi server

2. BIND TOKENS CRYPTOGRAPHICALLY TO SESSIONS

Anti-CSRF Token harus terikat secara kriptografis ke identitas sesi aktif
Mencegah eksploitasi cross-session token reuse dan pemalsuan token

3. DEFENSE IN DEPTH WINS

Kombinasikan Signed Token + SameSite Cookie + Fetch Metadata
Kekuatan sistem berlapis menjamin keamanan saat salah satu mekanisme gagal

Menjaga keamanan backend menuntut pemahaman arsitektural yang jelas. Jangan pernah mempercayai CORS untuk menjaga integritas database. Selalu ikat token CSRF ke ID sesi pengguna secara kriptografis, dan terapkan arsitektur pertahanan berlapis untuk menangkal berbagai vektor serangan lintas origin secara efektif.

CORS governs browser reads; cryptographic server tokens defend backend writes. Build multi-layered security.

Eksperimen kecil, interpretasi yang teliti.

SOP restricts cross-origin read; CSRF exploits cross-origin write. CORS relaxes read, it never defends write.

Software Engineering Lab 36 - CORS & CSRF: Mengamankan Backend

internal/cors • internal/csrf
internal/bank • cmd/demo
tests/integration_test.go • README.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/36-cors-and-csrf`

Lukman / lukman-ss

Sumber: commit 7ced81ed
Semua test PASS dengan Go 1.22 Race Detector aktif
Terbukti: CORS tidak mencegah mutasi data sisi server